Introduction to Quantum Computing for Visualization Researchers (IEEE VIS 2026 Tutorial)

Handout

Organizers
Hyeok Kim (KAIST), **Leilani Battle** (University of Washington), **Sara Mouradian** (University of Washington)

0. Disclaimer

There are multiple methods/paradigms for enabling, operating, and using quantum computers. Our goal is to introduce one of them: digital, superconductor-based quantum computing. It is a fast-evolving area, so what is in this material may change drastically in the future.

1. Some intuitions

A quantum program is a giant matrix multiplication (application-scale: thousands-millions of large-scale matrices).

In a sense that we cannot directly look into the inside and reconstruct it by forming a distribution,

- Composing a quantum program = constructing a Bayesian or deep learning model.
- Running a quantum program = obtaining a posterior predictive distribution.

→ Useful for problems that need to test lots of candidates.

2. Hardware

A quantum computer is a computing device that utilizes quantum mechanical properties:
superposition, **entanglement**, and **interference** of quantum particles.

Currently, quantum computers are made up of superconductors (IBM), photons (Xanadu Aurora), trapped ions (IonQ), neutral atoms, neutral molecules, and so on. More experimentations are going on this area as well.

A **Quantum Processing Unit** (QPU) is a chip that processes quantum operations, consisting of qubits.

A **qubit** (or quantum bit) is a basic information unit that exhibits superposition, entanglement, and interference.

3. State and Superposition

Many quantum computers materialize qubits as two-state (two-level) systems (simply put, a thing that can move between two states, typically 0 and 1).

State

A purely mathematical way to represent quantum elements.

Superposition

Qubits maintain those states in a probabilistic way.
In other words, a qubit state exists in a probabilistic distribution of a discrete outcome space (commonly, 0 and 1).
This stochastic property of qubit is called **superposition**.

However, we cannot directly look into such probabilities.
We need to measure them. But, once we measure them, we lose superposition. What an irony! So, we measure qubits multiple times to reconstruct the superposed states.

For example, measuring qubits will theoretically return the commensurate number of corresponding states.

In quantum, there are many mathematical things that do not make an immediate sense to real-world meanings. An important job in quantum computing is to build such mapping from real-world things to quantum properties.

State	Prob.	Counts
0	0.5	500
1	0.5	500
0	0.3	300
1	0.7	700

When measured 1,000 times (theoretical)

Mathematically, a qubit state is

Scalar (complex numbers)

$$|\psi\rangle = \alpha|0\rangle + \beta|1\rangle$$

$$= \begin{bmatrix} \alpha \\ \beta \end{bmatrix} \quad = \begin{bmatrix} 1 \\ 0 \end{bmatrix} \quad = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$$

$|\psi\rangle$ is read "ket psi".

Born Rule

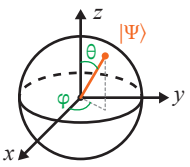
$$|\alpha|^2 + |\beta|^2 = 1$$

$$P(\psi=0) = |\alpha|^2$$
$$P(\psi=1) = |\beta|^2$$

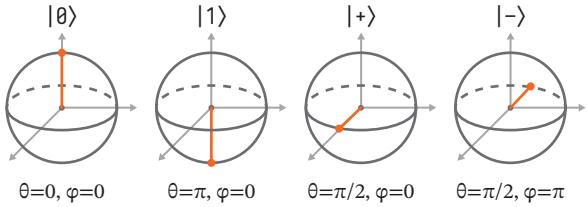
Common states in quantum computing

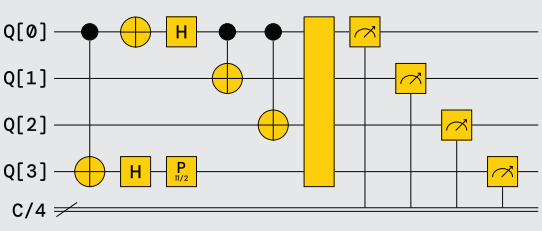
State	α	β	P(0)	P(1)
$\|0\rangle$	1	0	1	0
$\|1\rangle$	0	1	0	1
$\|+\rangle$	$1/\sqrt{2}$	$1/\sqrt{2}$	0.5	0.5
$\|-\rangle$	$1/\sqrt{2}$	$-1/\sqrt{2}$	0.5	0.5

Bloch sphere



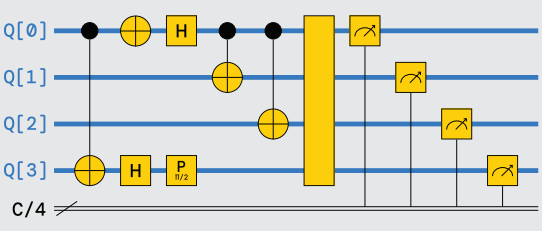
Bloch sphere representations for common states





4. Quantum circuit diagram

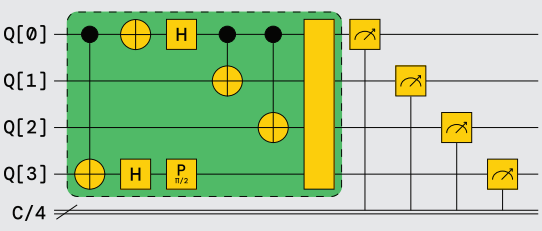
A traditional diagram for displaying quantum programs (or quantum circuits). Specifics may differ by toolkits, paradigms, and purposes.



Qubits

Represented by their indices and horizontal lines.

The qubit lines indicate to which qubit(s) operations are applied.

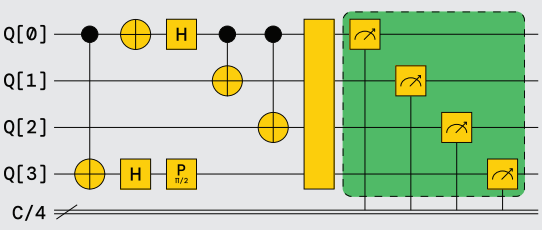


Operations (or gates)

Assigned on qubit lines where the operations are applied.

There are shared symbols for frequently used operations.

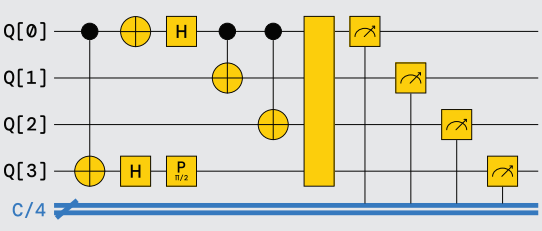
Less common ones tend to be shown as a box with acronyms.



Measurements

Typically as a variation of this gauge symbol.

In text-based ones (e.g., on a terminal), it is just a box with the letter M.



Classical bits

Where measured information is stored.

Typically indicated as the number of classical bits needed and a double line.

5. Quantum operations/gates

A gate manipulates a qubit's state (i.e., changing the alpha and beta values).

A gate must be expressed as a unitary matrix
of which the inverse and conjugate transpose are the same. $M^{-1}=M^{\dagger}$ "Dagger"

Original

$$M=\begin{bmatrix} a_0+b_0 i & a_1+b_1 i \\ a_2+b_2 i & a_3+b_3 i \end{bmatrix}$$

Inverse

$$M^{-1}=\mathrm{Det}(M)\begin{bmatrix} a_3+b_3 i & -a_1-b_1 i \\ -a_2-b_2 i & a_0+b_0 i \end{bmatrix}$$

Conjugate transpose

$$M^{\dagger}=\begin{bmatrix} a_0+b_0 i & a_2-b_2 i \\ a_1-b_1 i & a_3+b_3 i \end{bmatrix}$$

Example with a Y gate

$i \cdot i = -1$

$-i(-i) = -1$

$-i \cdot i = 1$

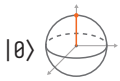
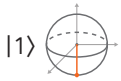
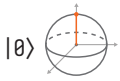
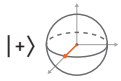
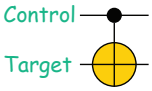
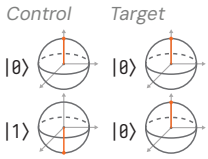
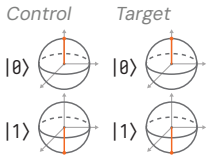
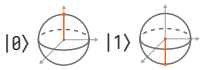
$$Y=\begin{bmatrix} 0 & -i \\ i & 0 \end{bmatrix}$$

$$Y^{-1}=\frac{1}{0-i(-i)}\begin{bmatrix} 0 & i \\ -i & 0 \end{bmatrix}=\begin{bmatrix} 0 & -i \\ i & 0 \end{bmatrix} \qquad Y^{\dagger}=\begin{bmatrix} 0 & -i \\ i & 0 \end{bmatrix}$$

$$Y^{-1}Y^{\dagger}=\begin{bmatrix} 0 & -i \\ i & 0 \end{bmatrix}\begin{bmatrix} 0 & -i \\ i & 0 \end{bmatrix}$$

$$=\begin{bmatrix} -i \cdot i & 0i-0i \\ 0i-0i & i(-i) \end{bmatrix}\begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}=I$$ Identity matrix

5.1. Common gates and their notations

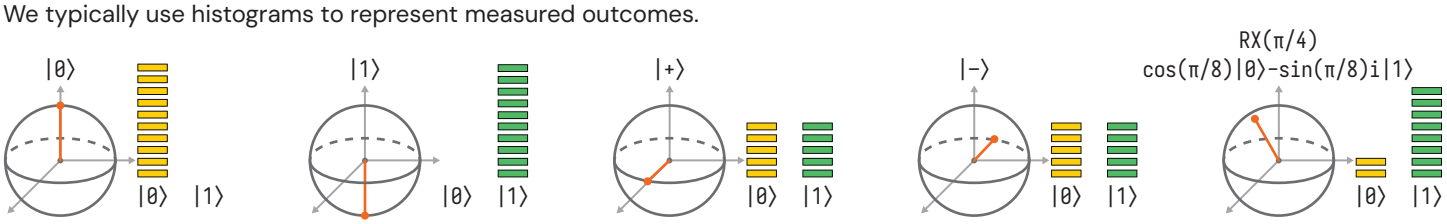
Name	Notation	Before	After	Description
X (Not)				Flips a qubit by the X axis (bit flip).
Hadamard (H)				Rotate a qubit by Π along the X and Z axes (or flip it against the Y axis). Turns |0⟩ into equal superposition.
C-NOT(CX, Control-Not, C-Not)				If the control qubit's state is |1⟩, then bit-flips the target qubit.
Swap				Swap the states of two qubits.

6. Measure

When we measure quantum particles (=when they are interacted with external energy), their superposed quantum states collapse (i.e., decoherence).

We can only measure a qubit through a certain axis (due to the uncertainty principle). In quantum computing, we typically use the Z axis (or the computational axis).

By measuring qubits, we can reconstruct qubits' superposed states.
Measurements usually happen at the end of the circuit, so we learn about the final superposed states.
We typically use histograms to represent measured outcomes.



To reconstruct the superposed states, we measure the same circuit multiple times (a few thousands).
Each run typically takes less than a few millisecond (depending on qubit materials and the number of operations).

7. Amplitudes, phase, and interference

We can express quantum superposed states in different ways; one of them is via amplitude and phase.
Amplitudes affect how qubits are measured;
Phases do not directly affect the measured outcomes, but they affect interference (which is often hidden).

$$|+\rangle = 1/\sqrt{2}|0\rangle + 1/\sqrt{2}|1\rangle$$

$$|-\rangle = 1/\sqrt{2}|0\rangle - 1/\sqrt{2}|1\rangle$$

Interference refers to the phenomenon that qubits' phases and amplitudes influence how qubits are superposed and measured. We can use interference to amplify or de-amplify qubit states to obtain desired outcomes.

In realistic scenarios, we can store by-product information in (relative) phases while changing the amplitudes of qubits. Then, at the end, we can rotate qubits in a way that switches phases to amplitudes, so we can measure that information.
How to utilize this is also a key to useful quantum algorithms.

8. Entanglement

When two qubits are **entangled**, we can infer about one qubit's state by looking at the other qubit's state.

If two qubits are equally entangled to have the same states (= **Bell state**), then we have the probability of 0.5 for measuring |00⟩ and |11⟩.

9. Logical and physical

Intuitively, **logical** elements are what humans think about quantum computing/information whereas **physical** elements are what machines work with.

9.1. Logical and physical circuits

- **Logical circuits** are quantum programs written by human programmers. They are also used in theoretical quantum computing.
- **Physical circuits** are those compiled for physical machines (i.e., QPUs), re-expressed as operations that a given QPU supports. A physical circuit for one QPU may not work on a different QPU if they have different sets of basic operations.
- **Compilation** (or **transpilation**) relies on many factors, including the fidelity of qubits and operations, qubit connectivity (whether qubit pairs are connected to enable multi-qubit operations), resource sharing (in the sense of cloud computing). Indeed, it is considered to be a hard problem, so compilation uses some randomization. Thus, each compilation technically results in a different physical circuit for the same logical circuit.
- Compilation is often referred to as **circuit optimization** because it involves optimization to maximize the success rate by minimizing the number of operations, minimizing the number of swap gates, maximizing the fidelity, etc.

9.2. Logical and physical qubits

- **Logical qubits** are those that we suppose as basic information units when we compose quantum programs. For example, when we have a C-NOT gate to two qubits in a circuit, then those two qubits are logical qubits.
- A single logical qubit is realized as a group of **physical qubits** in QPUs to enable more effective error correction.
- A caveat is that many commercially available QPUs only offer physical qubits. When we compile a circuit for an IBM QPU, then those qubits in the compiled circuit are physical qubits. It is a programmers' job to form logical qubits using physical qubits.
- How to form a logical qubit using multiple physical qubits is therefore an important research question in quantum computing.
- When visualizing physical qubits with operations, we need to consider a time dimension on top of 2-dimensional qubit layouts, which makes it a complicated visualization problem.

9.3. Logical and physical gates

- **Logical gates** are those written by humans.
- **Physical gates** are those supported by a selected QPU. They are decomposition of logical gates (initially), but they are merged and separated to ensure optimal performance (see above about circuit optimization).

10. No cloning Theorem

A quantum state cannot be copied.

11. Key application areas of quantum computing

- Security: providing ways to assess the strength of encryption methods through decryption algorithms
- Physics: simulating how quantum particles behave
- Chemistry: estimating energy levels and wave functions of chemical molecules
- Medicine: discovering novel medicines
- Logistics: searching optimal schedules and routes for logistics
- Other areas: telecommunication (routing network traffics), material science (discovering novel materials), finance (optimizing stock portfolios), quantum machine learning

12. Shor's algorithm for cryptography

12.1. Background

Modern cryptography relies on large prime numbers because it is highly costly to find prime factors of large numbers (e.g., 2048-bit = 2^{2048}). Shor's algorithm (Shor, 1994) is for finding the prime factor of an integer.

Modern standard for encryption is RSA-2048. It is known to take 300 trillion years with modern classical computers to crack. But, with quantum computers, it takes a few days with about 22-26,000 physical qubits (Cain et al., 2026; Pappalardo et al., 2026).

12.2. Overview

Essentially, Shor's algorithm is about guessing a number (as a factor) and verifying it. It has three main parts:
(1) modular exponentiation (to find periods), (2) quantum Fourier transform (QFT), and (3) computing factors. Among them, the first part is important to understand (the rest is quite established).

Shor, P. W. (1994). Algorithms for Quantum Computation: Discrete Logarithms and Factoring. In *the Proceedings of 35th Annual Symposium on Foundations of Computer Science*, 124-134, IEEE. DOI: 10.1109/SFCS.1994.365700.

Cain, M., Xu, Q., King, R., Picard, L. R. B., Levine, H., Endres, M., Preskill, J., Huang H-Y., & Bluvstein, D. (2026). Shor's Algorithm is Possible with as Few as 10,000 Reconfigurable Atomic Qubits. arXiv:2603.28627.

Pappalardo, A., Goudarzi, D., Hertz, V., Jacinto, H., Kroell, L., Schrottenloher, A., Shanahan, P., Gouzien, É., de La Vieuville, R., & Londe, V. (2026). Shor's Algorithm Compiled on Cat Qubits: 22593 Qubits for RSA-2048 in 3.14 Days. arXiv:2610.37839.

12.3. Definition

a = the number that we are guessing (we can try another number if it doesn't work).

r = the period of modular exponentiation function (iterated).

N = the number that we want to factor.

We want to find the smallest positive integer **r** such that

$$a^r \equiv 1 \pmod N$$

Interpretation: <**a** to the power of **r**> and **1** have the same remainder when divided by **N**.

Why do we do this? Solving this problem gives us some hints for the actual factorization.
Because a^r and **1** have the remainder when divided by **N**, **(a^r-1) can be divided by N and another integer.**

$$N \cdot k = a^r - 1$$
$$= (a^{r/2}-1)(a^{r/2}+1)$$

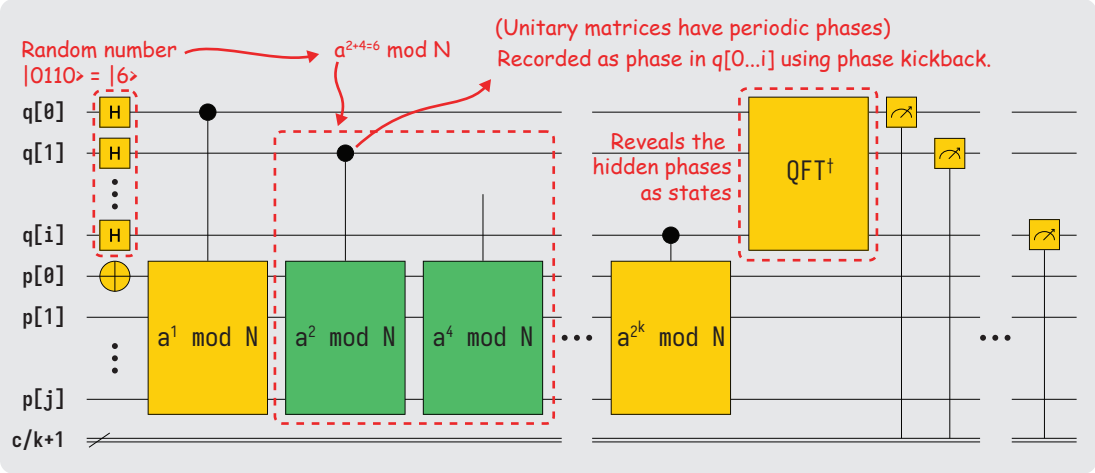
These numbers may not be the prime factors of **N**, but they are likely to be co-factors, which allows us to repeat this recursively.
Because we want integers, we want **r** to grow exponentially by 2.

Modular exponentiation?

$$c \equiv b^e \pmod m$$

Raising a base to an integer exponent and finding the remainder after dividing by a positive modulus.

12.4. Shor's algorithm circuit

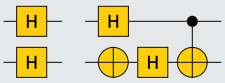


Quantum Fourier Transform is the quantum analogue of the discrete Fourier Transform (for separating hidden signals).

We then measure states (switched from phases)

Phase kickback refers to when controlled operations have effects on the control in addition to the targets in terms of the phase.

A quick example (use IBM Composer)



They will result in the same measurement outcome, but compare the statevectors of these two circuits.

12.4. Example

a = 2, the number that we are guessing.

r = the period of modular exponentiation function.

N = 15, the number that we want to factor.

Estimated phases Source: Qiskit

Reads backward

	Register Output	Phase		Phase Fraction	Guess for r
0	00000000(bin) = 0(dec)	0/256 = 0.00	0	0.00	0/1	1
1	01000000(bin) = 64(dec)	64/256 = 0.25	1	0.25	1/4	4
2	10000000(bin) = 128(dec)	128/256 = 0.50	2	0.50	1/2	2
3	11000000(bin) = 192(dec)	192/256 = 0.75	3	0.75	3/4	4

Added for precision

Most frequent candidate for r = 4.
$(a=2)^{(r/2=2)} - 1 = 4 - 1 = 3$
$(a=2)^{(r/2=2)} + 1 = 4 + 1 = 5$

12.5. Notes

- We don't know a generalized method for encoding modular exponentiation.
- To effectively factorize application-scale large integers, we need a few thousands of qubits (current commercial availability: ~150; 2000+ qubits expected in 2030-ish).

13. Grover's search

13.1. Background

In theoretical computer science, we can classify problems in the following categories

- Search: to find an answer
- Decision: to determine if a candidate is an answer
- Optimization: to approximate an answer (because we cannot get the answer directly)
- Counting: to count the number of answers

Quantum search algorithms tend to build on top of decision problems.

13.2. Overview

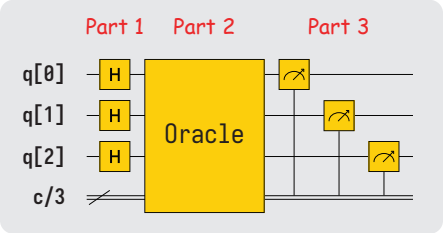
Grover's search can solve problems that we can constitutes the correctness of answers but it is intractable to search correct candidate answers directly (i.e., we can determine an answer but can't find). Example applications include satisfiability problems and hashed key recovery problems.

Part 1. Preparing "unknown" distribution by uniformly superposing qubits (= a bunch of Hadamard gates).

Part 2. Encoding an oracle that amplify amplitudes toward correct answers. How to define an oracle depends on a given problem. It can be a hash function, a logic operator, etc.

Part 3. Recovering the amplitude by measuring so that high-frequency candidates are likely answers. We need to run $\Pi/4\sqrt{N}$ times, where N s the number of items being searched. $N = 2^n$, where n = # qubits.

13.3. Grover's search circuit



13.4. Notes

- We need more work on useful oracle methods for application-scale problems.
- Currently, many example tutorials include very simple cases (e.g., string match).
- To do problems like unstructured database search, oracles must be able to associate each candidate value with the corresponding index (represented as qubits), which needs quantum RAMs (QRAMs).
  QRAMs are currently at a prototype stage (4-8 qubits).

14. Quantum Approximate Optimization Algorithm (QAOA)

14.1. Background & overview

QAOA is a representative quantum optimization algorithm. QAOA finds a certain string that maximizes (or minimizes) the cost. Here, a string can be a network, a Boolean function, etc., and cost can be the utility, benefit, difficulty, etc., of that string.

- Quantum optimization is about finding the ground state energy. The **ground state energy** of a physical system is its unique energy level (at the absolute zero temperature, or 0° Kelvin).
- We use **Hamiltonian** matrix to express the total energy of a physical system.
- Some gates have **parameters**. For example, RX(Π/2) rotates a qubit by Π/2 (90 degrees) by the X axis. Here, Π/2 is a parameter for the RX gate.
- **Quantum Adiabatic Process.**
  When a quantum system evolves slowly enough,

  the system will adapt

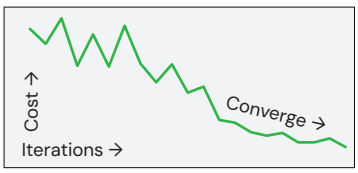
  to the environment's ground state energy.
- **For quantum computing...**
  If there are sufficient gaps between operations,

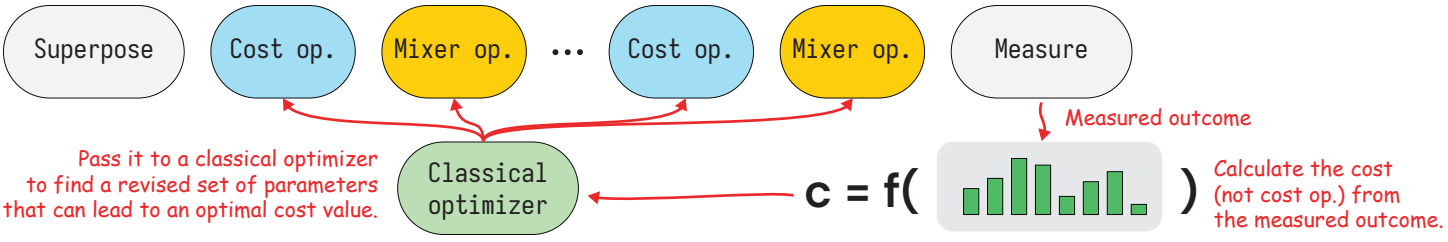
  the states of qubits will approximate
  represented as gate parameters
  to those of the solutions.
  because our QPUs are cooled at near 0°K.
- Problem = physical system
- Solution = ground state
- We express our problem as a physical system in a way that its ground state energy is the desired answer of the problem.

14.2. How QAOA works

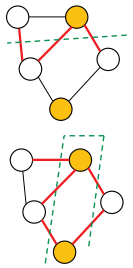
First, we have two systems (or two Hamiltonians)

- Problem system as **cost operator**: its ground state is the solution.
- A simple system as **mixer operator**: without this, the problem system fails to change due to quantum mechanics (i.e., enables the quantum adiabatic process).
- Then, we repeat the below process, and get the final parameters as the cost converges.





14.3. Example: MAX CUT



Finding a bi-partition of nodes that maximizes the number of edges (cuts) crossing between the two groups. There is no known polynomial time solution for this problem (NP-hard).

We can map a MAX CUT problem to QAOA as below.

- Cost operator: edges in a graph
- Mixer operator: a bunch of X gates
- Cost function (utility function): the number of cuts

14.4. Other applications of QAOA

Logistics, telecommunication, stock portfolio…

14.5. Variational Quantum Eigen Solver (VQE)

VQE also uses quantum adiabatic process and looks for ground state energy. Compared to QAOA (interested in using the final optimized parameters), VQE is interested in the actual ground state energy of a chemical molecule. In other words, we want the optimized parameters out of a VQE process because those parameters represent the molecule's wavefunction that exhibits its ground state energy. This matters because ground state energy tells about how molecules behave (e.g., when they react with others, when they stay stable, etc.), which is highly important in chemistry.

15. Quantum Error Correction and Mitigation

15.1. Noisy intermediate-scale quantum computing (NISQ)

QPUs with not too many qubits (~1,000) that are not yet fault-tolerant. Currently, widely available commercial quantum computers have about 25-150 qubits. They exhibit non-negligible amount of errors.

For example, IBM's Boston QPU has 156 qubits. When checked August 14, 2026, 00:00 am PT, the calibrated X gate error for Qubit 156 was 0.00426, which means 4.3 errors every 1,000 operations. To compare, 5-node MAX CUT QAOA has a few thousand physical operations.

15.2. Quantum Error Correction (QEC)

Error correction itself is not new to quantum. Classical bits also have errors. Semiconductor chips (for classical computers) tend to have less than one bit flip error per trillion operations. Classical computers still fix those errors.

Error correction is basically a majority vote (repetition code). We run the same operations N times (typically N = 3). Then, the majority outcome is the correct value. 0/0/0 → keep 0. 1/1/0 → change to 1.

In quantum computing, we correct two types of errors: bit flip errors and phase flip errors.

- Bit flip errors: errors along the Z axis.
- Phase flip errors: errors along the X and Y axes.

15.3. How do we correct quantum errors?

We measure additional qubits (or ancilla qubits) and based on the measured outcomes, we correct as needed. We can use the final measurement or mid-circuit measurement.

Mid-circuit measurement measures qubits in the middle of the circuit and correct them as needed. This is more preferred because it can actively fix error in the middle of execution.

But, as said earlier, qubits collapse after measurements. So, we recover the qubits' phases and states using quantum teleportation and feed-forward operations.

We need additional qubits for error correction. To fix both bitflip and phase flip errors, we need 9 (= 3 • 3) qubits. This is called 9-qubit Shor code. This implies that we need a few more physical qubits than the problem needs logically. There are other quantum correction codes: 9-qubit Shor code, 7-qubit Stean code, 5-qubit code, etc. It is a highly active research area with new approaches (almost everyday).

15.4. Quantum Error Mitigation

Error mitigation is about post-processing of measured outcomes by looking at the fidelity information of the QPU and using some classical algorithmic methods.

Mitigation aims to reduce noise in measurement outcomes and does not require additional physical qubits.

We still need error correction though because error mitigation has scalability concerns (the outcome space grows exponentially).